
Data Structures

BS (CS) _Fall_2025

LAB 02 Tasks



Learning Objectives:

1. Class Templates with unit testing (Gtest)

Lab Overview

This lab focuses on understanding and implementing classes using templates in C++ along with unit testing using Google Tests.

Task 0: Simple Generic Holder

Create a template class `Holder<T>` that stores a single item of any type.

Data Members:

- **T item:** the stored value.

Functions:

- **Constructor:** initialize the item.
- **setItem(T value):** set the stored item.
- **getItem():** return the stored item.
- **displayType():** prints the data type of the stored item using `typeid(item).name()` from `typeinfo` header.

Test Cases:

- Create `Holder<int>`, `Holder<double>`, and `Holder<string>`.
- Set and get values for each.
- Print stored type and value.

Task 1:

Define a template class **Container** for storing different type of values e.g. int, double, float etc. This container would keep same type of values/elements at a time. The **Container** class should consist of the following member variables:

T * values; A pointer to set of values it contains. This member points to the dynamically allocated array (which you will be allocating in constructor).

capacity; An integer that shows total capacity of the container

counter; An integer counter variable which increments upon every insertion operation and decrements upon removal of any element; representing total number of elements currently present in the container.

The container should consist of following functions:

- **constructor** with capacity as parameter
- **isFull** which will return true if counter reaches capacity otherwise false
- **insert** which will receive a value of some type (int/double/float) and insert into the container if it's not already full
- **search** which will return true if the container contains value passed as parameter

- **remove** which will remove a value, if it exists, from the container and shifts all subsequent values one index back.
- **print** which will print all values contained in the container

Now create a test class to create three instances of Container for types: **int**, **float**, **double** and call all functions separately for each instance. Your output must be properly formatted.

Task 2:

Create a template class **MyMatrix** for performing various matrix operations with different data types e.g. int, double, float etc. This matrix class will handle same type of elements at a time and provide comprehensive matrix functionality. The **MyMatrix** class should consist of the following member variables:

T **matrix** A pointer to pointer (T**) representing a 2D dynamically allocated array to store matrix elements. **rows** An integer representing the number of rows in the matrix **cols** An integer representing the number of columns in the matrix

The MyMatrix class should consist of the following functions:

Constructor with rows and columns as parameters. This constructor should dynamically allocate memory for a 2D array and initialize all elements to zero.

Copy Constructor to create a deep copy of another MyMatrix object to avoid shallow copying issues.

Destructor to properly deallocate the dynamically allocated memory to prevent memory leaks.

- **inputMatrix()** which will take input from user to fill the matrix elements. This function should prompt the user to enter values row by row.
- **printMatrix()** which will display the matrix in a properly formatted manner with appropriate spacing between elements.
- **getElement(int row, int col)** which will return the element at specified position with proper bounds checking.
- **setElement(int row, int col, T value)** which will set the element at specified position with proper bounds checking.
- **isSquare()** which will return true if the matrix is a square matrix (rows == cols), otherwise false.
- **multiplyMatrix(const MyMatrix& other)** which will return a new MyMatrix object containing the product of current matrix and the matrix passed as parameter. This function should verify that number of columns of first matrix equals number of rows of second matrix.
- **scalarMultiply(T scalar)** which will return a new MyMatrix object where each element is multiplied by the scalar value passed as parameter.

- **getDeterminant()** which will calculate and return the determinant of the matrix. This function should only work for square matrices and should handle 1x1, 2x2, and 3x3 matrices. For larger matrices, display appropriate message.
- **isSymmetric()** which will return true if the matrix is symmetric ($A = A^T$), otherwise false. This function should first check if matrix is square.
- **getTrace()** which will return the sum of diagonal elements. This function should only work for square matrices.
- **fillRandom(T min, T max)** which will fill the matrix with random values between the specified minimum and maximum range.

Test Cases:

Create instances for different data types (**int, float, double**) and perform the following operations for each type:

1. Create matrices of different sizes (2x2, 3x3, 4x3)
2. Input values or fill with random numbers
3. Display the matrices using printMatrix()
4. Check isSquare() functionality with both square and non-square matrices
5. Demonstrate matrix multiplication operations
6. Calculate and display determinant for square matrices
7. Test symmetry checking
8. Calculate trace for square matrices
9. Demonstrate scalar multiplication